

Lab Assessment Rubrics of Digital System Design (CPE-344)

Lab Number		07								
Lab Title		Introduction to behavioral modeling and Implementation of Decoders and Comparator circuits using behavioral modeling.								
Submitted by										
Names		Registration Number								
Sumaira Safeer		FA22-BCE-019								
Rubrics name & number							Marks			
							In-Lab	Post-Lab		
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.									
	R3: Interpretation of Subject Knowledge: Ability to interpret and explain mathematical and/or visual forms, including equations, diagrams, graphics, figures, and tables.									
ProblemAnalysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.									
Design	R7: Implementing Design Strategy: Ability to execute a solution takinginto consideration design requirements and pertinent contextual elements. [Block Diagram/Flow chart/Circuit Diagram]									
	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.									
Modern ToolsUsage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.									
	R11: Tools Evaluation: Ability to simulate the experiment and them using hardware tools to verify The results.									
Individualand Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.									
	R13: Management of Teamwork: Ability to appreciate, understand and work with multidisciplinary team members.									
Rubrics to follow										
Rubrics	R2	R3	R5	R6	R7	R8	R9	R11	R12	R13
In-Lab										
Post-Lab										

Lab No: 07

Title:

Introduction to behavioral modeling and Implementation of Decoders and Comparator circuits using behavioral modeling.

1. Introduction 2. Procedural assignments 3. Conditional statement 4. Case statement 5. Looping statements

Lab Objectives:

Implementation of different combinational circuits using behavioral modeling:

Task #01:

Implement 2x4, 3x8, and 4 x 16 decoder using dataflow modeling / conditional operators.

A decoder is a combinational logic circuit that has 'n' input signal lines and 2^n output lines.

2x4 decoder (Dataflow Modeling):

Truth table for a 2:4 decoder

A	B	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

From this truth table, we get the following logic equations for the outputs.

$$Y0 = A'B'$$

$$Y1 = A'B$$

$$Y2 = AB'$$

$$Y3 = AB$$

Verilog Code:

```
//=====
// 2x4 Decoder - Dataflow Modeling
// Author: Senior Verilog Engineer
//=====
module decoder_2x4 (
```

```

input wire [1:0] in, // 2-bit input
input wire en, // Enable signal
output wire [3:0] out // 4-bit output
);

// Dataflow modeling using continuous assignment
assign out[0] = en & (~in[1]) & (~in[0]);
assign out[1] = en & (~in[1]) & in[0];
assign out[2] = en & in[1] & (~in[0]);
assign out[3] = en & in[1] & in[0];

endmodule

```

Verilog Test Bench:

```

//=====
// Testbench for 2x4 Decoder
//=====
`timescale 1ns/1ps
module tb_decoder_2x4;

    reg [1:0] in;
    reg en;
    wire [3:0] out;

    // Instantiate the decoder
    decoder_2x4 uut (
        .in(in),
        .en(en),
        .out(out)
    );

    initial begin
        $display("Time\tEN\tIN\tOUT");
        $monitor("%0dns\t%b\t%b\t%b", $time, en, in, out);

        // Test sequence
        en = 0; in = 2'b00; #10;
        en = 1; in = 2'b00; #10;
        in = 2'b01; #10;
        in = 2'b10; #10;
    end
endmodule

```

```

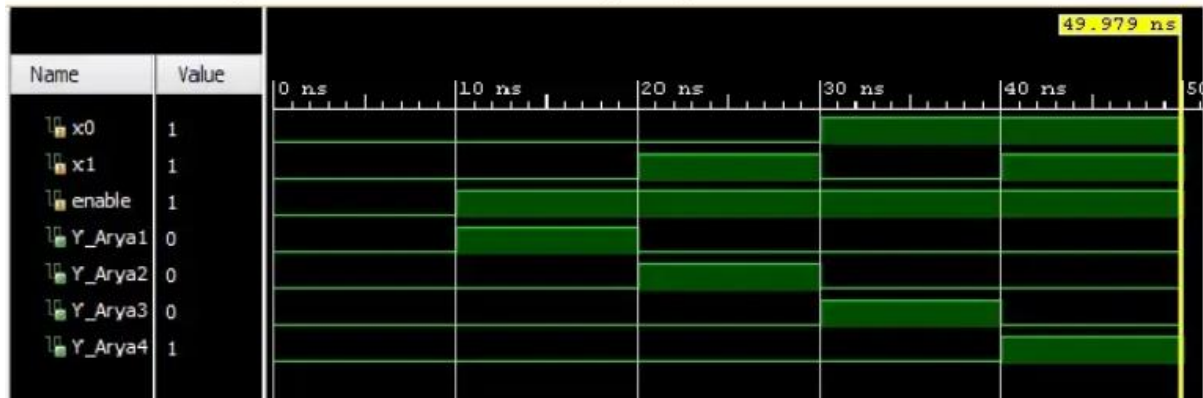
in = 2'b11; #10;
en = 0; in = 2'b10; #10;

$finish;
end

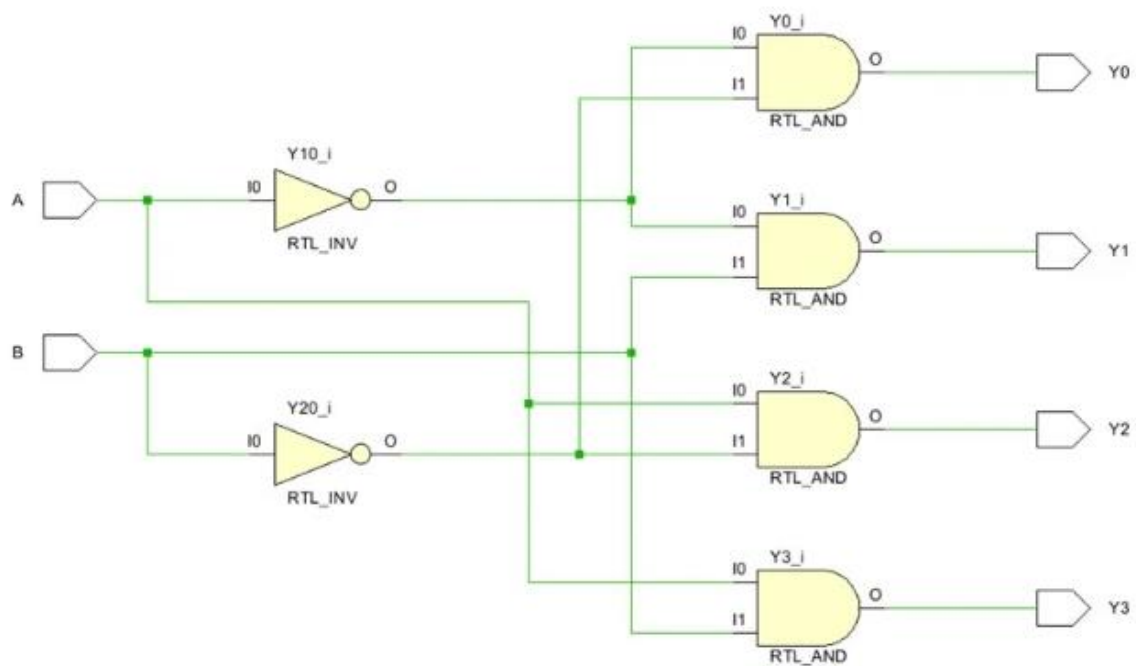
endmodule

```

Simulation Waveform:



RTL Viewer Diagram:



3-to-8 Decoder (Dataflow Modeling)

Verilog Code:

```
// 3-to-8 Decoder using Dataflow Modeling
module decoder3to8 (
    input wire [2:0] in, // 3-bit input
    input wire en, // Enable signal
    output wire [7:0] out // 8-bit output
);

    // Dataflow modeling: assign statement with enable control
    assign out = (en) ? (8'b00000001 << in) : 8'b00000000;

endmodule
```

Verilog Test Bench:

```
`timescale 1ns / 1ps

module tb_decoder3to8;

    reg [2:0] in;
    reg en;
    wire [7:0] out;

    // Instantiate the decoder
    decoder3to8 uut (
        .in(in),
        .en(en),
        .out(out)
    );

    initial begin
        // Monitor signals
        $monitor("Time=%0t | en=%b | in=%b | out=%b", $time, en, in, out);

        // Test sequence
        en = 0; in = 3'b000; #10; // Disabled
        en = 1; in = 3'b000; #10;
        in = 3'b001; #10;
        in = 3'b010; #10;
        in = 3'b011; #10;
    end
endmodule
```

```

        in = 3'b100; #10;
        in = 3'b101; #10;
        in = 3'b110; #10;
        in = 3'b111; #10;

        // Disable again
        en = 0; in = 3'b101; #10;

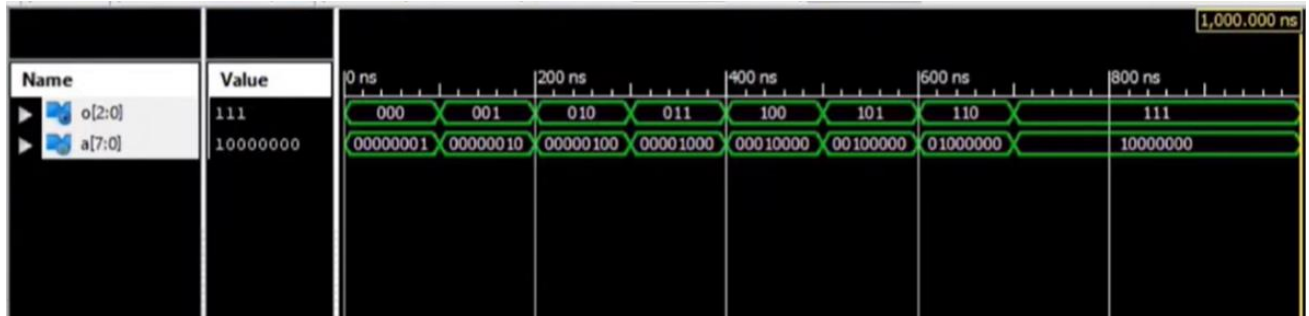
        $stop; // End simulation
    end

endmodule

```

Simulation

Waveform:



Task #02:

Implement 2x4, 3x8, 4x16, 5x32 decoder using behavioral modeling.

3 to 8 Decoder using behavioral modeling:

Verilog Code:

```

// File: decoder3x8.v
// 3-to-8 Decoder using Behavioral Modeling
module decoder3x8 (
    input wire [2:0] din, // 3-bit binary input
    input wire      en, // Enable signal
    output reg [7:0] dout // 8-bit one-hot output
);

always @(*) begin
    if (en) begin
        // Default all outputs to 0
        dout = 8'b00000000;
    end
end

```

```

        // Activate the selected output line
        case (din)
            3'b000: dout = 8'b00000001;
            3'b001: dout = 8'b00000010;
            3'b010: dout = 8'b00000100;
            3'b011: dout = 8'b00001000;
            3'b100: dout = 8'b00010000;
            3'b101: dout = 8'b00100000;
            3'b110: dout = 8'b01000000;
            3'b111: dout = 8'b10000000;
            default: dout = 8'b00000000; // Safety
        endcase
    end else begin
        dout = 8'b00000000; // Disabled state
    end
end
endmodule

```

Verilog Test Bench:

```

// File: tb_decoder3x8.v
`timescale 1ns/1ps

module tb_decoder3x8;

    reg [2:0] din;
    reg      en;
    wire [7:0] dout;

    // Instantiate the decoder
    decoder3x8 uut (
        .din(din),
        .en(en),
        .dout(dout)
    );

    initial begin
        // Monitor changes
        $monitor("Time=%0t | EN=%b | DIN=%b | DOUT=%b", $time, en, din,
            dout);
    end
endmodule

```

```

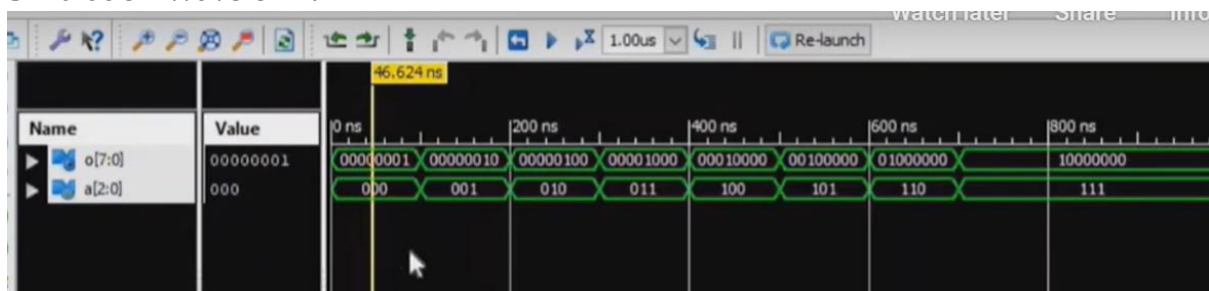
// Test sequence
en = 0; din = 3'b000; #10;
en = 1; din = 3'b000; #10;
din = 3'b001; #10;
din = 3'b010; #10;
din = 3'b011; #10;
din = 3'b100; #10;
din = 3'b101; #10;
din = 3'b110; #10;
din = 3'b111; #10;
en = 0; din = 3'b101; #10;

$stop; // End simulation
end

endmodule

```

Simulation Waveform:



Wave Description:

RTL Viewer Diagram:

Task #03: Implement 2-bit comparator using behavioral modeling.

Verilog Code:

```

// 2-bit Comparator using Behavioral Modeling
// Compares two 2-bit numbers A and B
// Outputs: A_gt_B (A > B), A_eq_B (A == B), A_lt_B (A < B)

module comparator_2bit (
    input [1:0] A, // 2-bit input A

```

```

input [1:0] B, // 2-bit input B
output reg A_gt_B, // High if A > B
output reg A_eq_B, // High if A == B
output reg A_lt_B // High if A < B
);

// Behavioral description using always block
always @(*) begin
    // Default values
    A_gt_B = 0;
    A_eq_B = 0;
    A_lt_B = 0;

    if (A > B)
        A_gt_B = 1;
    else if (A == B)
        A_eq_B = 1;
    else
        A_lt_B = 1;
    end

endmodule

```

Verilog Test Bench:

```

`timescale 1ns / 1ps

module comparator_2bit_tb;

    reg [1:0] A;
    reg [1:0] B;
    wire A_gt_B;
    wire A_eq_B;
    wire A_lt_B;

    // Instantiate the comparator
    comparator_2bit uut (
        .A(A),
        .B(B),
        .A_gt_B(A_gt_B),
        .A_eq_B(A_eq_B),

```

```

        .A_lt_B(A_lt_B)
    );

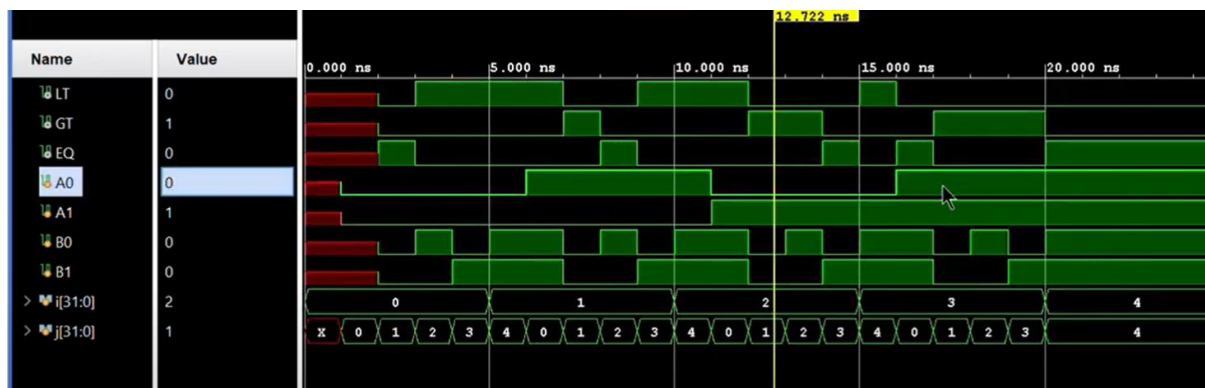
    initial begin
        // Monitor outputs
        $monitor("Time=%0t | A=%b B=%b | A_gt_B=%b A_eq_B=%b A_lt_B=%b",
            $time, A, B, A_gt_B, A_eq_B, A_lt_B);

        // Test all combinations
        A = 2'b00; B = 2'b00; #10;
        A = 2'b00; B = 2'b01; #10;
        A = 2'b01; B = 2'b00; #10;
        A = 2'b10; B = 2'b10; #10;
        A = 2'b11; B = 2'b01; #10;
        A = 2'b01; B = 2'b11; #10;
        A = 2'b11; B = 2'b11; #10;

        $stop; // End simulation
    end
endmodule

```

Simulation Waveform:



Task #04: Design a 4-bit Comparator and implement in FPGA and see the results.

Verilog Code:

```

module comparator_4bit_bh(
    output reg EQ,
    output reg GT,

```

```

output reg LT,
input [3:0] a,
input [3:0] b
);

always @(*) begin
    LT = (a < b);
    EQ = (a == b);
    GT = (a > b);
end

endmodule

```

Verilog Test Bench:

```

module comparator_tb;
    // Declare wires for outputs
    wire EQ;
    wire GT;
    wire LT;
    // Declare registers for inputs
    reg [3:0] a;
    reg [3:0] b;
    // Instantiate the comparator module
    comparator_4bit_bh dut(EQ,GT,LT,a,b);

    initial begin
        // Test scenario for greater than
        a = 5;
        b = 3;
        #10; // Wait for 10 time units
        // Display the inputs and outputs
        $display("a = %d, b = %d, EQ = %b, GT = %b, LT = %b", a, b, EQ, GT, LT);

        // Test scenario for less than
        a = 3;
        b = 5;
        #10; // Wait for 10 time units
        // Display the inputs and outputs
        $display("a = %d, b = %d, EQ = %b, GT = %b, LT = %b", a, b, EQ, GT, LT);
    end
endmodule

```

```

// Test scenario for equal to
a = 4;
b = 4;
#10; // Wait for 10 time units
// Display the inputs and outputs
$display("a = %d, b = %d, EQ = %b, GT = %b, LT = %b", a, b, EQ, GT, LT);

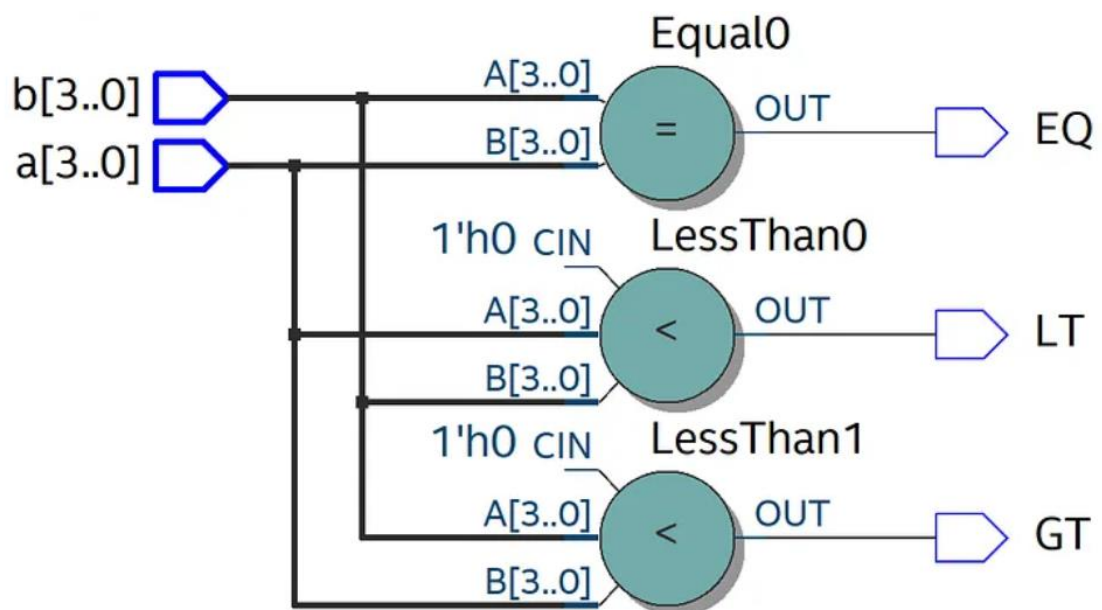
// Finish the simulation
$finish;
end
endmodule

```

Simulation Waveform:



RTL Viewer:



100%

RTL Diagram of 4-Bit Comparator